

Primary sources & further reading

Part 09 — 📄 Model Context Protocol · 09-mcp/papers/PAPERS.md · 09-mcp/resources/RESOURCES.md

Papers & Primary Sources — Model Context Protocol

MCP is a living spec, not a paper — the primary sources here are the specification itself and its changelogs, not a single canonical publication. Read the spec pages first; the academic papers are independent security analysis of the deployed ecosystem, useful for grounding `notes/02`'s claims in something other than the spec authors' own assurances.

The canon (read in this order)

#	Source	What it is	Link
1	★ Introducing the Model Context Protocol — Anthropic	The original November 2024 announcement: the problem (information silos, bespoke integrations), the M×N framing, and the initial ecosystem partners	https://www.anthropic.com/news/model-context-protocol
2	★ MCP Specification (latest)	The normative spec: data layer, transport layer, primitives, authorization. Always read <code>/specification/latest</code> , not a pinned old version, unless you specifically need history	https://modelcontextprotocol.io/specification/latest
3	Architecture overview	The Host/Client/Server model, the data-layer/transport-layer split, and a full worked JSON-RPC example (discovery, tool call, notifications) under the current spec	https://modelcontextprotocol.io/docs/2026-07-28/learn/architecture
4	Security Best Practices	Confused deputy, token passthrough, SSRF, state handle hijacking, local server compromise, OAuth URL validation — the attack classes covered in <code>notes/02 §2</code>	https://modelcontextprotocol.io/specification/2026-07-28/basic/security_best_practices
5	2026-07-28 changelog	The stateless-protocol redesign: handshake removal, <code>server/discover</code> , per-request <code>_meta</code> , sampling/logging deprecation	https://modelcontextprotocol.io/specification/2026-07-28/changelog
6	2025-11-25 changelog	The revision the SDK in this module actually negotiates by default: OAuth CIMD, icons metadata, experimental Tasks, tool-naming guidance	https://modelcontextprotocol.io/specification/2025-11-25/changelog

SDKs and reference implementations

Repo	What's inside	Link
Python SDK	The <code>mcp</code> package this module's code uses (<code>mcp>=2.2.0</code>) — server, client, both transports	https://github.com/modelcontextprotocol/python-sdk
TypeScript SDK	The Tier-1 JS/TS implementation; takes Zod schemas directly for tool input validation	https://github.com/modelcontextprotocol/typescript-sdk
Reference servers	Anthropic-maintained example servers (filesystem, fetch, memory, git, ...) — read these before writing your own from scratch	https://github.com/modelcontextprotocol/servers
MCP Inspector	The official dev tool for poking at a server interactively without writing a client	https://github.com/modelcontextprotocol/inspector
MCP Registry (service)	The registry backend and <code>server.json</code> metadata format spec	https://github.com/modelcontextprotocol/registry

Academic security analysis (independent of the spec authors)

Paper	Year	One-line takeaway	Link
Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions	2025	A threat taxonomy across four attacker types (malicious developer, external attacker, malicious user, security flaw) spanning 16 threat scenarios — the broadest early survey	arXiv:2503.23278
MCP Safety Audit: LLMs with the Model Context Protocol Allow Major Security Exploits — Radosevich & Halloran	2025	Demonstrates models can be steered into using legitimate MCP tools for code execution, unauthorized remote access, and credential theft; ships <code>MCPSafetyScanner</code> , an agentic auditing tool	arXiv:2504.03767
Enterprise-Grade Security for the Model Context Protocol (MCP): Frameworks and Mitigation Strategies	2025	Practitioner-oriented mitigation framework aimed at enterprise deployments rather than a single exploit class	arXiv:2504.08623
Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers	2025	Static analysis of 1,899 open-source servers; finds 8 vulnerability classes, only 3 overlapping traditional software CWEs	arXiv:2506.13538
A First Look at the Security Issues in the Model Context Protocol Ecosystem	2025	Large-scale registry study (67,057 servers across 6 registries); introduces <code>MCPInspect</code> , flags 833 vulnerable servers and 18 with suspicious/deceptive descriptions	arXiv:2510.16558
Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning	2026	STRIDE/DREAD threat model across Host/Client/LLM/Server/Data-store/AuthZ-server; identifies tool-description poisoning as the most impactful client-side vector	arXiv:2603.2489

How to read these (20 minutes, 2 passes)

1. **Spec pages:** don't read linearly — use them as reference. Read the architecture overview once fully, then treat the rest as lookup material when `notes/01/`/`notes/02` cite a specific mechanism.

2. **Security papers:** read the abstract and the vulnerability taxonomy/table first. These papers move fast and their specific CVE-style findings will age; the taxonomies (what *categories* of thing can go wrong) age much better than any specific vulnerable-server count.

Numbers (server counts, download counts, vulnerability counts) in any of the above will be stale within months — treat them as evidence a problem class is real and material, not as current statistics to quote.

Resources — Model Context Protocol

Official documentation

Resource	Why	Link
MCP documentation home	Canonical docs: introduction, architecture, build-server/build-client guides	https://modelcontextprotocol.io/
Specification (latest)	The normative spec — data layer, transport layer, authorization	https://modelcontextprotocol.io/specification/latest
MCP Blog	Where spec revisions, roadmap posts, and adoption/ecosystem updates actually get announced	https://blog.modelcontextprotocol.io/
Build servers guide	Step-by-step server construction per language, current spec revision	https://modelcontextprotocol.io/docs/2026-07-28/develop/build-server
Build clients guide	The client-side equivalent — what <code>code/mcp_client_demo.py</code> in this module is a minimal version of	https://modelcontextprotocol.io/docs/2026-07-28/develop/build-client

SDK repositories

Repo	What's inside	Link
python-sdk	The <code>mcp</code> PyPI package used by this module's code	https://github.com/modelcontextprotocol/python-sdk
typescript-sdk	The Tier-1 TS/JS SDK; Zod-native schemas	https://github.com/modelcontextprotocol/typescript-sdk
servers (reference implementations)	Official filesystem/fetch/memory/git servers — the best "read real code" starting point	https://github.com/modelcontextprotocol/servers
inspector	Interactive dev tool for testing a server without writing a client	https://github.com/modelcontextprotocol/inspector
modelcontextprotocol/modelcontextprotocol	The spec's own source repo — changelogs, SEPs (spec enhancement proposals), governance docs	https://github.com/modelcontextprotocol/modelcontextprotocol

Server discovery

Resource	What it gives you	Link
Official MCP Registry	The authoritative, verified-namespace index of publicly available servers	https://registry.modelcontextprotocol.io/
Registry API docs	REST API for programmatic server discovery — useful if you're building a client that federates many servers	https://registry.modelcontextprotocol.io/docs

Client/host support (where you'll actually use this)

- Claude (Desktop, Code, API Connectors) — <https://claude.com/docs/connectors/building>
- ChatGPT / OpenAI API MCP support — <https://developers.openai.com/api/docs/mcp/>
- VS Code MCP servers — <https://code.visualstudio.com/docs/copilot/chat/mcp-servers>
- Cursor MCP support — <https://cursor.com/docs/context/mcp>

Cheat sheets

- JSON-RPC 2.0 spec (the wire format underneath everything) — <https://www.jsonrpc.org/specification>
- Also see `../../_shared/cheatsheets/`

Communities

- MCP GitHub Discussions — the working-group/SEP conversations that shape the spec: <https://github.com/modelcontextprotocol/modelcontextprotocol/discussions>
- Anthropic Developer Discord — has an active MCP channel for server-building questions
- `r/mcp` and `r/ClaudeAI` — informal but active for "why isn't my server showing up" debugging

Where this connects back

- `../..06-ai-agents/resources/RESOURCES.md` — tool-calling and agent frameworks that consume MCP servers as one of their tool sources.
- `../..13-ai-security/` — OWASP Top 10 for LLM Applications, for the general prompt-injection background this module's security notes assume.
- `../..16-ai-tech-stack/tracks/claude-code/resources/RESOURCES.md` — the practical side: configuring MCP servers inside Claude Code day to day.